

Traceable Secret Sharing:

Strong Security and Efficient Constructions

Presented by: Sebastian David Albu

Original authors: Dan Boneh, Aditi Partap, and Lior Rotem

Modern Public Key Cryptography, SS 2025

Motivation

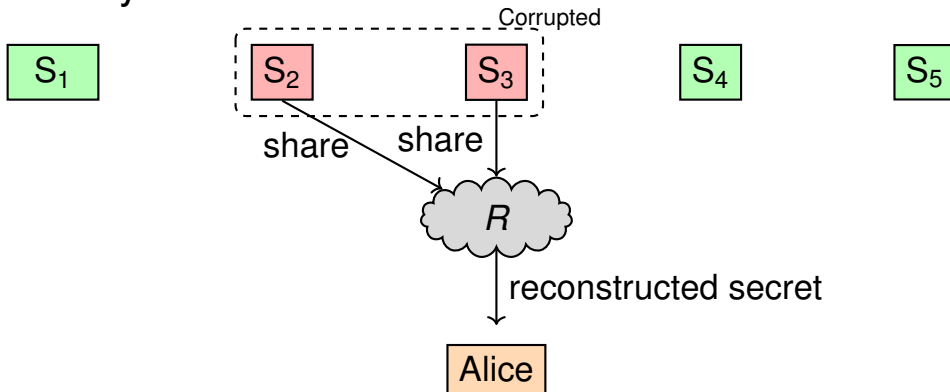
Motivation: Traceable Secret Sharing

- **Secret Sharing:** In threshold secret-sharing schemes, a secret s is split into n shares so that any t shares suffice for reconstruction, while any $t - 1$ shares reveal no information.
- **The problem:** If a subset of servers colludes and sells their shares, the owner (Alice) might have no way to hold them accountable.
- **Our goal:** Introduce a mechanism to trace the leaked secret shares back to the corresponding servers, thus deterring such leakage.

Traceability in Action

- **Setup:** Alice splits secret s with a t -out-of- n scheme.
- **Attack:** A group of $f < t$ corrupted servers colludes. They build a “reconstruction box” R :
 - Encodes their f shares inside a sealed module.
 - When given any further $t - f$ honest shares, R outputs s .
- **Sale of R :** These servers can sell or leak R as a standalone device. The buyer only needs to supply honest share(s) to recover s .
- **Consequence:** Without tracing, Alice cannot identify which server-shares ended up in R .

Traceability in Action



The presented traceable secret sharing scheme allows Alice to extract identifying information from reconstruction box R , thereby linking it to the corrupted servers.

Definitions and Overview

Definition: Traceable (Threshold) Secret Sharing Scheme

- **Share**($1^\lambda, s, n, t, \rho$) outputs for each party a share sh_i , a tracing key component tk_i , and a verification key component vk_i .
- **Rec**($sh_{i_1}, \dots, sh_{i_t}$) reconstructs the secret s from any t shares.
- **Trace** takes the tracing key ($tk_1, \dots, tk_n$) and, given black-box access to a reconstruction box R , outputs a set of indices (a subset of parties) and a proof.
- **Verify** uses the verification keys ($vk_1, \dots, vk_n$), the alleged traitor set, and the proof to confirm the correctness of the tracing.

Security of Traceable Secret Sharing Schemes

Security of Traceable Secret Sharing Schemes

A **traceable secret-sharing scheme** needs to satisfy three security properties:

- Secrecy
- Traceability
- Non-imputability

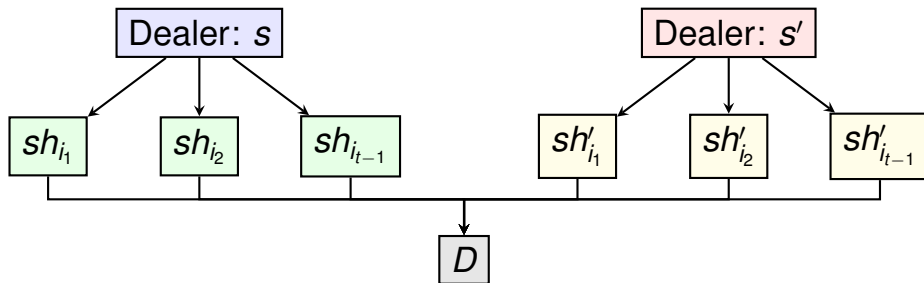
Secrecy in Traceable Secret Sharing Schemes

The classic **secrecy** property for secret-sharing schemes ensures that any subset of less than t parties learns nothing about the secret s . In our notation, let

$$\text{TTSS} = (\mathbf{Share}, \mathbf{Rec}, \mathbf{Trace}, \mathbf{Verify})$$

be a traceable secret-sharing scheme for secrets from a space $\mathcal{S} = \{S_\lambda\}_{\lambda \in \mathbb{N}}$.

Visualization of Secrecy



Adversary D cannot distinguish the distributions
of these $t - 1$ shares beyond $\epsilon(\lambda)$

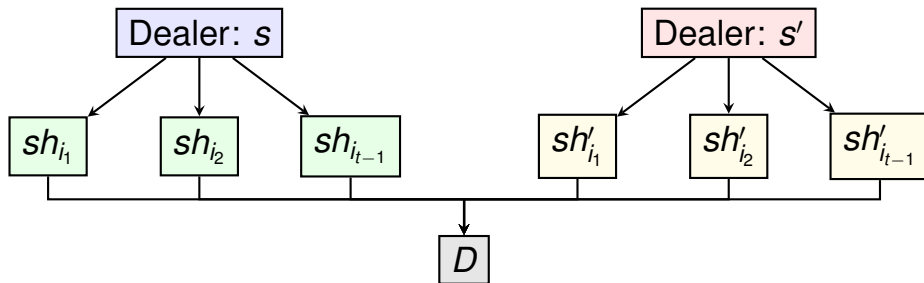
Visualization of Secrecy

- In the diagram, the left side shows shares from a sharing of s (in blue/green) while the right side shows shares from s' (in red/yellow).
- An adversary D that receives only $t - 1$ of these shares cannot distinguish whether they originated from s or s' beyond a negligible advantage $\epsilon(\lambda)$.

(For guessing if “yes” or “no”, the advantage must be ≥ 0.5).

$$\left| \Pr \left[D(sh_{i_1}, \dots, sh_{i_{t-1}}) = 1 \right] - \Pr \left[D(sh'_{i_1}, \dots, sh'_{i_{t-1}}) = 1 \right] \right| \leq \epsilon(\lambda)$$

Visualization of Secrecy



Adversary D cannot distinguish the distributions
of these $t - 1$ shares beyond $\epsilon(\lambda)$

Tracer Secrecy

- For traceable secret-sharing schemes, it makes sense to require that secrecy is maintained even against the tracer.
- That is, the tracing key tk does **not** help in breaking secrecy.

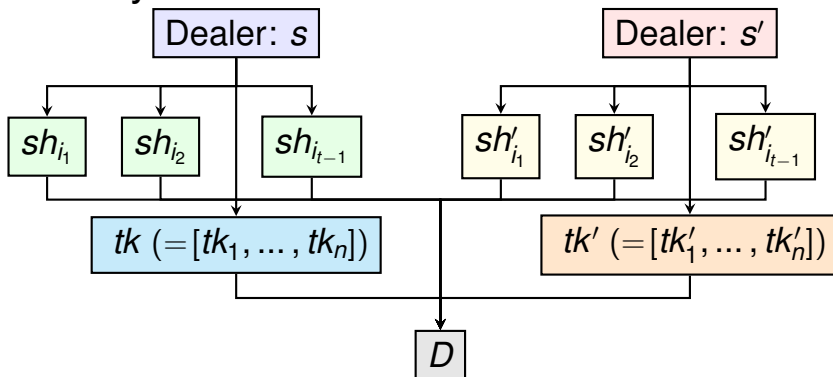
Tracer Secrecy

In a traceable secret-sharing scheme $\text{TTSS} = (\mathbf{Share}, \mathbf{Rec}, \mathbf{Trace}, \mathbf{Verify})$, **tracer secrecy** ensures that the tracing keys tk do not help an adversary learn any information about the secret. In other words, even if a PPT distinguisher D is given:

$$D(tk, sh_{i_1}, \dots, sh_{i_{t-1}})$$

it cannot tell whether the shares come from a sharing of s or s' (for any two secrets) beyond a negligible advantage $\epsilon(\lambda)$.

Tracer Secrecy

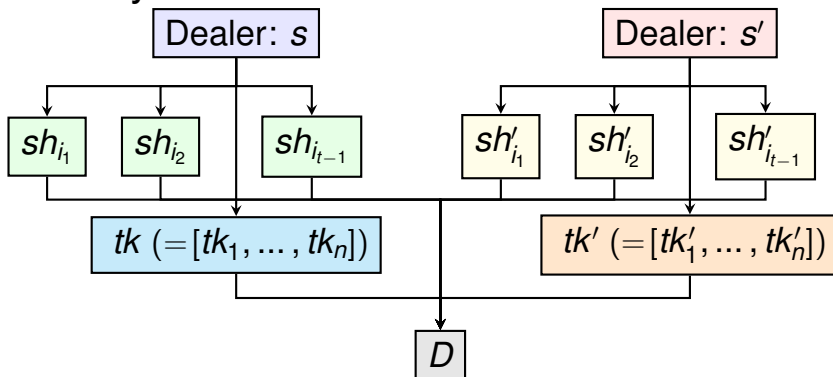


D cannot distinguish the input distribution beyond $\epsilon(\lambda)$

Tracer Secrecy

- Here, two parallel sharing processes produce (shares, tracing keys) for s and s' respectively.
- Even when the distinguisher D sees both the shares and the full tracing key, it is unable to learn anything useful about s versus s' .

Tracer Secrecy



D cannot distinguish the input distribution beyond $\epsilon(\lambda)$

Traceability and Good Reconstruction Boxes

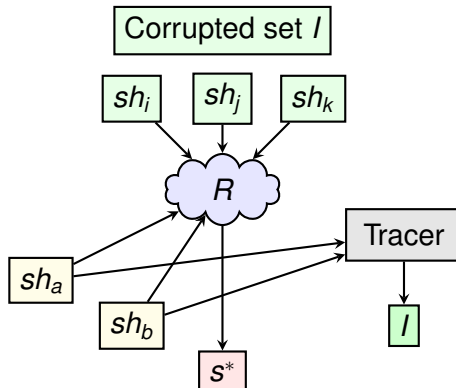
Traceability:

- A traceable secret-sharing scheme ensures that if a coalition $I \subseteq [n]$ of size $f < t$ colludes to create a reconstruction box R (which combines their shares), then R should, when given $t - f$ additional correct shares, output the secret s^* .
- Moreover, there is a **tracing algorithm** that, given only black-box access to R , can identify at least one of the parties in I .

Traceability and Good Reconstruction Boxes

- **Good Reconstruction Boxes:** A reconstruction box R is said to be $(n, t, f, s^*, \rho, \epsilon)$ -good if, when it is given $t - f$ new shares (generated from the same correlation string ρ and secret s^*), it outputs the correct secret s^* with probability at least ϵ .
- **Intuitive Idea:** The tracer can query R using additional shares (from honest parties) and, by analyzing R 's responses, can deduce information about which parties must have been involved in the construction of R .

Traceability and Good Reconstruction Boxes

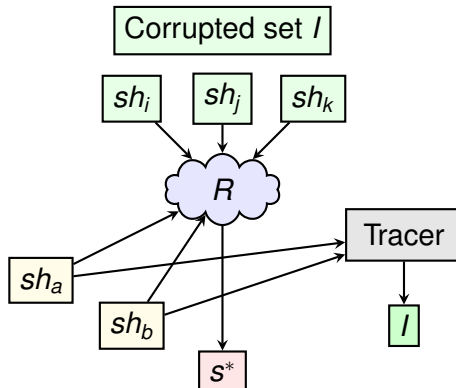


R is $(n, t, f, s^*, \rho, \epsilon)$ -good if it outputs s^*
with probability at least ϵ when given $t - f$ new shares.

Traceability and Good Reconstruction Boxes

- In this diagram, a coalition I (green boxes) forms a reconstruction box R .
- By querying R with additional honest shares (yellow boxes), the secret s^* is correctly output.
- The tracer uses these responses to link R back to I (flagged on the right), thereby enforcing **traceability**.

Traceability and Good Reconstruction Boxes



R is $(n, t, f, s^*, \rho, \epsilon)$ -good if it outputs s^*
with probability at least ϵ when given $t - f$ new shares.

Non-Imputability

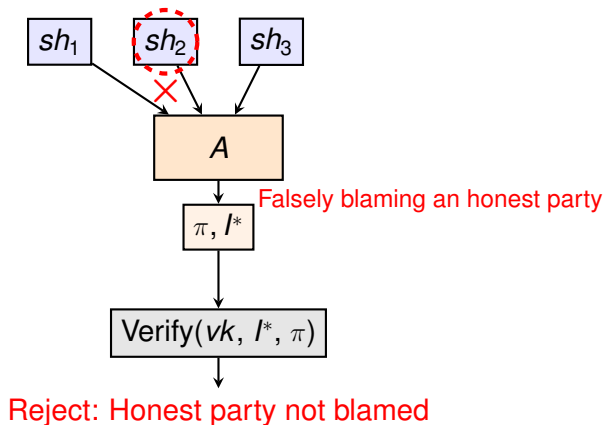
Intuition:

- Non-imputability guarantees that an adversary (even with access to the tracing keys tk and verification keys vk) cannot produce a proof that falsely accuses an honest party.
- That is, even a malicious tracer cannot blame a party that did not leak its share.

Informal Definition:

- A traceable secret-sharing scheme satisfies non-imputability if for every PPT adversary A , the probability that A can output a proof π and a subset I^* which includes an honest party i^* (who did not participate in the leak) and passes verification is negligible in the security parameter λ .

Non-Imputability

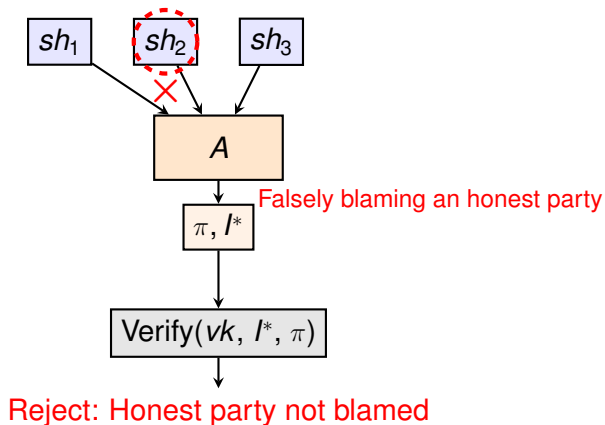


In a secure scheme, even with tk and vk , a malicious adversary cannot produce a valid proof blaming an honest party.

Non-Imputability

- In this diagram, three honest parties (blue) provide their shares and keys to the adversary A .
- Even if A produces a proof (π, I^*) and claims that one of the honest parties is to be blamed, the verification process (using vk) will reject the proof if any honest party is wrongly implicated.
- Thus, the probability of successful, false blame (non-imputability violation) is negligible.

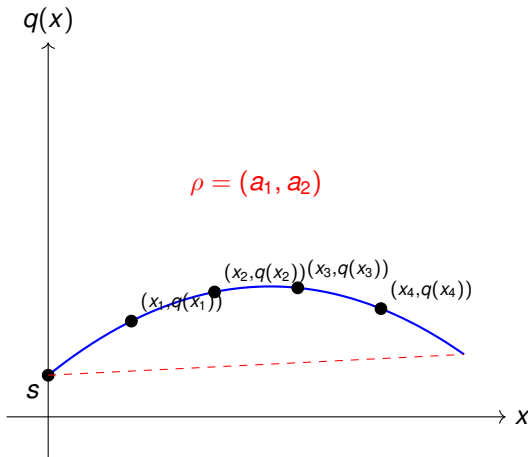
Non-Imputability



In a secure scheme, even with tk and vk , a malicious adversary cannot produce a valid proof blaming an honest party.

Traceable Secret Sharing with Shamir Secret Shares

Visualizing Shamir Secret Sharing (Degree 2)



Traceable Shamir Secret Sharing: Overview

Standard Shamir Secret Sharing:

- The dealer selects a random polynomial

$$q(x) = s + a_1x + \cdots + a_{t-1}x^{t-1},$$

over a finite field $\mathbb{F}$, where the free coefficient s is the secret.

- Each party $i \in [n]$ is assigned an evaluation point x_i and receives the share

$$sh_i = (x_i, q(x_i)).$$

- Any t shares can reconstruct $q(x)$ (via Lagrange interpolation) while any $t - 1$ shares reveal nothing about s .

Tracing via Polynomial Interpolation and Factorization

Overview:

- Consider a Shamir secret-sharing scheme with threshold t .
- Assume that $f = t - 1$ parties are corrupted and their shares $\{(x_i, y_i = q(x_i))\}_{i=1}^{t-1}$ are hardcoded into a reconstruction box R .
- The reconstruction box R expects an additional share (x_t, y_t) as input and outputs the secret s reconstructed via Lagrange interpolation.

Example: Setup

We are in $\mathbb{F}_7$. The dealer picks a random polynomial of degree $t - 1 = 2$ whose constant term is the secret $s = 3$ and $\rho = (a_1, a_2) = \{2, 1\}$:

$$q(X) = s + a_1X + a_2X^2 = 3 + 2X + X^2 \pmod{7}.$$

Share Generation: Choose distinct nonzero evaluation points $x_1 = 1, x_2 = 2, x_3 = 3$. Compute:

$$y_1 = q(1) = 3 + 2 \cdot 1 + 1 \cdot 1^2 = 6,$$

$$y_2 = q(2) = 3 + 2 \cdot 2 + 1 \cdot 2^2 = 11 \equiv 4,$$

$$y_3 = q(3) = 3 + 2 \cdot 3 + 1 \cdot 3^2 = 18 \equiv 4.$$

Each party P_i receives the share (x_i, y_i) together with a tracing key x_i .

Example: Reconstruction Box

Parties P_1 and P_2 (indices 1 and 2) collude and build a black-box program R (reconstruction box) that hardcodes their shares $(1, 6)$ and $(2, 4)$.

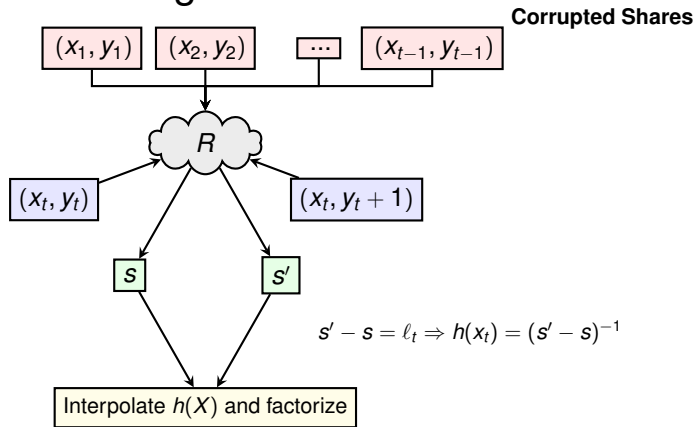
On input a candidate share (x^*, y^*) , R outputs the secret by Lagrange interpolation:

$$R(x^*, y^*) = \ell_1(x^*)6 + \ell_2(x^*)4 + \ell_3(x^*)y^*,$$

where

$$\ell_3(x^*) = \prod_{j=1}^2 \frac{x^* - x_j}{x_3 - x_j} = \frac{x^* - 1}{2} (x^* - 2) \pmod{7}.$$

Visualizing the Tracing Procedure



Roots $\{x_1, \dots, x_{t-1}\}$ identify the corrupted parties

Tracing Idea:

- **Correct Reconstruction:** With input share (x_t, y_t) , R outputs

$$s = \sum_{i=1}^{t-1} \ell_i(x_1, \dots, x_{t-1}, x_t) y_i + \ell_t(x_1, \dots, x_{t-1}, x_t) y_t,$$

where ℓ_i are the Lagrange coefficients.

- **Perturbed Input:** Now, feeding R the modified share $(x_t, y_t + 1)$ yields an output

$$s' = \sum_{i=1}^{t-1} \ell_i y_i + \ell_t (y_t + 1).$$

Example: Lagrange Coefficients

The goal is to recover the tracing polynomial

$$h(X) = \prod_{k=1}^2 \frac{x_k - X}{x_k} = \frac{(1-X)(2-X)}{1 \cdot 2} = \frac{(X-1)(X-2)}{2} = \ell_3(X) \pmod{7},$$

whose roots $X = 1, 2$ identify the colluders.

Since $\deg(h) = 2$, we need 3 evaluations $(x^*, h(x^*))$ to interpolate.

Because

$$\delta = R(x^*, y^* + 1) - R(x^*, y^*) = \ell_3(x^*),$$

we directly recover

$$h(x^*) = \delta.$$

Example: Extracting the Tag at Honest Point

- Use honest share (3, 4):

$$s = R(3, 4) = 3, \quad s' = R(3, 5) = 3 + \ell_3(3) \cdot 1.$$

- Compute

$$\ell_3(3) = \frac{3-1}{2} (3-2) = 1 \cdot 1 = 1 \pmod{7}.$$

- Thus $\delta_1 = s' - s = 1$, so

$$h(3) = 1.$$

Example: Extracting the Tag at Fresh Points

- At $x^* = 4$: pick $y^* = 0$

$$s = R(4, 0), \quad s' = R(4, 1) = s + \ell_3(4) \cdot 1,$$

$$\ell_3(4) = \frac{4-1}{2} (4-2) = \frac{3}{2} \cdot 2 = 3 \pmod{7} \Rightarrow h(4) = 3.$$

- At $x^* = 5$:

$$\ell_3(5) = \frac{5-1}{2} (5-2) = 2 \cdot 3 = 6 \pmod{7} \Rightarrow h(5) = 6.$$

- Collected:

$$(x^*, h(x^*)) = (3, 1), (4, 3), (5, 6).$$

Tracing Idea:

- **Difference and Reconstruction:** Subtracting, we get:

$$s' - s = \ell_t(x_1, \dots, x_{t-1}, x_t).$$

By algebraic manipulation, one can show that

$$\ell_t = \prod_{k=1}^{t-1} \frac{x_k}{x_k - x_t}.$$

Taking reciprocals,

$$h(x_t) \triangleq \prod_{k=1}^{t-1} \frac{x_k - x_t}{x_k} = (s' - s)^{-1}.$$

Tracing Idea:

- **Interpolation & Factorization:** Repeating this experiment with multiple fresh inputs for x_t , the tracer obtains several evaluations $\{(x'_t, h(x'_t))\}$ of the degree- $t - 1$ polynomial

$$h(X) = \prod_{k=1}^{t-1} \frac{x_k - X}{x_k}.$$

Since the roots of $h(X)$ are exactly the x -values of the corrupted parties, the tracer can use polynomial interpolation followed by factorization to recover the roots, and hence trace the corrupted parties.

Example: Interpolation: Setting Up

We have

$$h(x) = ax^2 + bx + c \quad \text{and} \quad (x, h(x)) \in \{(3, 1), (4, 3), (5, 6)\} \pmod{7}.$$

Plugging in gives the system of congruences:

$$\begin{cases} 9a + 3b + c \equiv 1, \\ 16a + 4b + c \equiv 3, \\ 25a + 5b + c \equiv 6 \end{cases} \pmod{7}.$$

Reducing coefficients mod 7:

$$\begin{cases} 2a + 3b + c = 1, \\ 2a + 4b + c = 3, \\ 4a + 5b + c = 6. \end{cases}$$

Example: Interpolation: Solving for a, b, c

1. Subtract eqn 1 from eqn 2:

$$(2a + 4b + c) - (2a + 3b + c) = 3 - 1 \implies b = 2.$$

2. Subtract eqn 2 from eqn 3:

$$(4a + 5b + c) - (2a + 4b + c) = 6 - 3 \implies 2a + b = 3 \Rightarrow 2a = 1 \Rightarrow a = 4.$$

3. Plug $a = 4, b = 2$ into eqn 1:

$$2 \cdot 4 + 3 \cdot 2 + c = 1 \implies 8 + 6 + c \equiv 1 \implies 14 + c \equiv 1 \implies c = 1.$$

Example: Interpolation: Final Polynomial & Roots

Thus

$$h(x) = 4x^2 + 2x + 1 \pmod{7}.$$

To identify colluders, solve $h(x) \equiv 0 \pmod{7}$:

$$4x^2 + 2x + 1 \equiv 0 \pmod{7}.$$

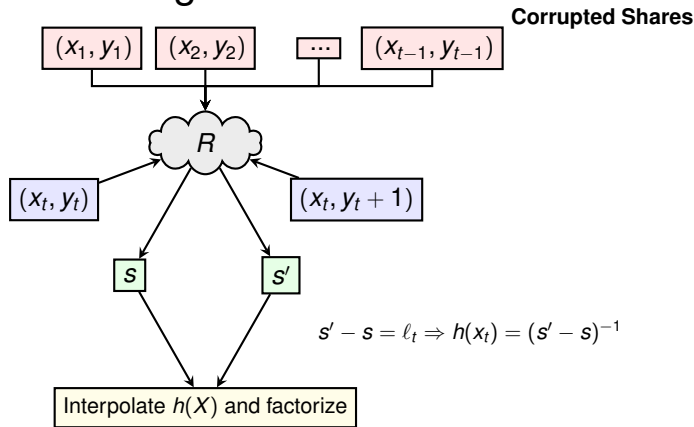
Check to find traitor's x :

$$h(1) = 4 + 2 + 1 = 7 \equiv 0, \quad h(2) = 16 + 4 + 1 = 21 \equiv 0.$$

Hence the roots are $x = 1, 2$, matching the two corrupted servers.

Also, $x = 3$ gives $h(3) = 43 \pmod{7} \equiv 1$, correctly showing that party 3 was not part of the corruption!

Visualizing the Tracing Procedure



Roots $\{x_1, \dots, x_{t-1}\}$ identify the corrupted parties

Traceable Shamir: Sharing Algorithm (Share)

The **Share** algorithm takes as inputs

- the security parameter 1^λ
- a secret $s \in \mathbb{F}$
- integers n and t (with t being the reconstruction threshold)
- a correlation string ρ interpreted as $t - 1$ coefficients

Traceable Shamir: Sharing Algorithm (Share)

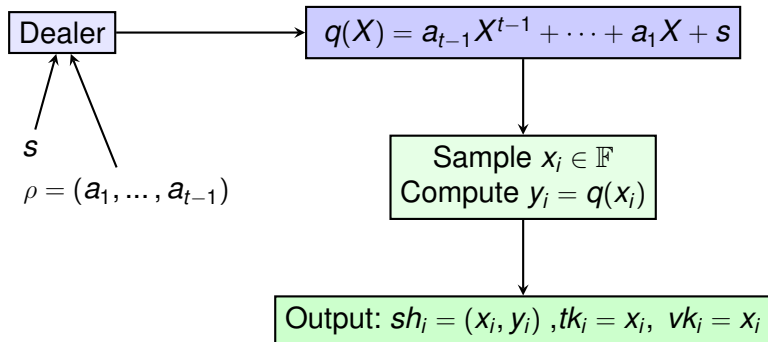
1. Parse ρ as $(a_1, \dots, a_{t-1}) \in \mathbb{F}^{t-1}$. Hence, the polynomial

$$q(X) = a_{t-1}X^{t-1} + \dots + a_1X + s$$

has degree $t - 1$ and $q(0) = s$.

2. Sample a random x_i from $\mathbb{F}$.
3. Compute $y_i = q(x_i) = s + \sum_{j=1}^{t-1} a_j x_i^j$.
4. Set the share $sh_i = (x_i, y_i)$ and also set the tracing key $tk_i = x_i$ and verification key $vk_i = x_i$.
5. Output (sh_i, tk_i, vk_i) .

Traceable Shamir: Sharing Algorithm (Share)



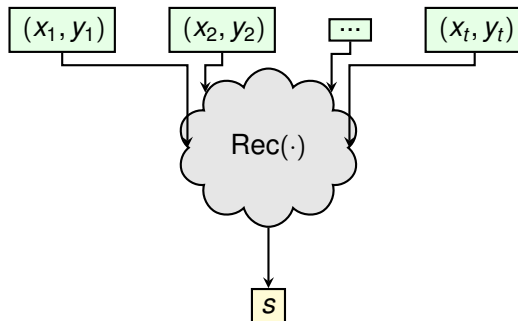
The share sh_i (with tracing and verification keys) is produced using random evaluation of the polynomial $q(X)$.

Traceable Shamir: Reconstruction Algorithm (Rec)

The reconstruction algorithm, **Rec**, is the standard Shamir reconstruction. Given t shares $\{sh_i = (x_i, y_i)\}_{i=1}^t$ (with distinct x_i values), the secret is recovered by computing:

$$s = \sum_{i=1}^t \left(\prod_{\substack{k=1 \\ k \neq i}}^t \frac{x_k}{x_k - x_i} \right) y_i.$$

Traceable Shamir: Reconstruction Algorithm (Rec)



Using Lagrange interpolation on the t shares,
the secret s is reconstructed.

Conclusion

Conclusion and Contributions

- **Accountable Secret Sharing:** Introduced a Shamir-based TTSS scheme that preserves t -out-of- n secrecy while enabling tracability of any reconstruction box built by up to $t - 1$ colluders.
- **Traceability via Black-Box Access:** Showed how the Trace procedure recovers at least one traitor by querying the reconstruction box on perturbed shares and extracting Lagrange-coefficient “fingerprints.”
- **Security Guarantees:**
 - Secrecy: Coalitions of $< t$ learn nothing about the secret.
 - **Traceability:** Any malicious $f < t$ colluders are identifiable. (At least one colluder!)
 - **Non-Imputability:** A malicious tracer cannot frame honest parties. (Not applicable in every scheme they presented)
- **Practical Implications:** Enables accountable threshold cryptosystems and robust distributed storage with deterrence against secret-leakage collusion.

Questions?

Traceable Secret Sharing:

Strong Security and Efficient Constructions

Presented by: Sebastian David Albu

Original authors: Dan Boneh, Aditi Partap, and Lior Rotem

Modern Public Key Cryptography, SS 2025